

Master Thesis Overview of Code Base -

For the Master Thesis of Felix Wohlert

1 Introduction

This codebase implements the heterogeneous-agent neoclassical (HANC) housing model of my thesis: an overlapping-generations, small-open-economy general-equilibrium model in which households make a discrete *region* $\times$ *tenure* housing choice (rural/urban, rent/own) alongside a continuous consumption–savings decision. The household problem is solved with DC-EGM; the general equilibrium and transition dynamics use the sequence-space (fake-news) Jacobian method. The model is built on the `EconModel` and `GEModelTools` packages, and the central object is the class `HANCHousingModelClass`.

1.1 Key objects (namespaces)

Every model instance carries six namespaces (declared in `settings()`):

- `.par` — **Parameters**. All calibrated structural parameters (preferences β, ρ, γ ; production α, δ ; housing shares μ_1, μ_2 ; regional land X_u, X_r ; LTV, transaction costs, $\dots$), grid sizes (`Na`, `Nz`, `Nh`, `T`, `J`), and the precomputed grids (asset grid, productivity process `z_grid` / `z_trans`, life-cycle earnings profile `chi`).
- `.ss` — **Steady state**. The scalar value of every model variable in long-run equilibrium, plus the stationary distribution `ss.D`, the age-cohort distribution `ss.D_cohort/ss.D_birth`, and `ss.cohort_weights`.
- `.path` — **Transition path**. For every variable, a time series of shape $(T, 1)$ (widened to $(T, ncols)$ during vectorised Jacobian evaluation). `path.D` holds the (T, Nz, Nh, Na) distribution path.
- `.ini` — **Initial conditions**. The $t = 0$ state a transition starts from: lagged aggregates and the initial distribution. For a permanent transition these come from the

initial steady state, while the Jacobian is built around the *terminal* steady state.

- **.sol — Household solution.** Policy-function arrays on the regular grids: `sol.a` (savings), `sol.c` (consumption), `sol.pr_choices` (4-way housing-choice probabilities), `sol.pr_urban`. Shape (Nfix, Nz, Nh, Na).

1.2 Other framework concepts

- **Blocks / DAG.** The equilibrium is a directed acyclic graph of block functions (listed in `settings()`); each reads inputs and writes outputs in topological order.
- **Unknowns / targets.** The GE solver searches for the unknown paths (`H_u`, `H_r`) that zero the target residuals (`clearing_H_u`, `clearing_H_r`).
- **Shocks.** Exogenous driving paths: `Gamma`, `L_supply`, `hh_scale`, `kappa`, `r_world`.
- **HH inputs / outputs.** The household block consumes prices `inputs_hh = [r, w, q_u, q_r, f_u, f_r, kappa]` and returns aggregates `outputs_hh → A_hh, C_hh, H_u_hh, H_r_hh`.
- **Jacobians.** `jac_hh` is the household sequence-space Jacobian; `H_U` is the system Jacobian of targets w.r.t. unknowns. Both feed the Broyden solver.
- **Housing states** (Nh = 4): 0 = rural renter, 1 = urban renter, 2 = rural owner, 3 = urban owner.
- **Closure.** Small open economy: `r_world` is exogenous, the firm FOC $rK = r_{\text{world}} + \delta$ pins capital, the asset-market clearing $A = A_{hh}$ is dropped, and net foreign assets NFA are the residual degree of freedom.

2 HANCHousingModel.py

Defines the model class and the grid/parameter setup.

- `_rouwenhorst_transition(n, rho)` — builds the Rouwenhorst transition matrix for the AR(1) productivity process; returns an $n \times n$ matrix.
- `_stationary_distribution(P)` — iterates a Markov matrix to its ergodic distribution; returns a length- n vector.
- `HANCHousingModelClass.settings()` — declares the namespaces, household grids/policies/inputs, the shock/unknown/target lists, and the DAG block ordering.

- `HANCHousingModelClass.setup()` — sets all baseline structural parameters, grid sizes, and the open-economy closure parameters.
- `HANCHousingModelClass.allocate_GE()` — allocates `ss/path` arrays for every GE variable by scanning the block signatures (custom; bypasses framework assertions).
- `HANCHousingModelClass.allocate()` — creates grids and allocates the household policy and distribution arrays.
- `HANCHousingModelClass.create_grids()` — builds the asset grid, the Rouwenhorst productivity process, and the life-cycle earnings profile `chi` read from `lifeearnings.xlsx`.
- `HANCHousingModelClass.evaluate_blocks(...)` — evaluates the full DAG along the path (wrapper into `transition.py`).
- `HANCHousingModelClass.configure_open_economy(r_world_path, ...)` — sets the small-open-economy closure and the exogenous world-rate path.
- `_set_unknowns(x)` / `_get_errors()` / `_evaluate_H(x)` — map the solver vector to/from the path arrays and return target residuals.
- `compute_jacs(dx)` — computes all model Jacobians (wrapper into `transition.py`).
- `_set_shocks(...)` — writes AR(1) shock paths onto path.
- `_set_unknowns_ss()` / `_set_ini(...)` — initialise the path / initial-condition namespaces at the steady state.

3 `blocks.py`

The DAG-ordered general-equilibrium block functions. All are Numba-compiled and write their outputs in place; path arrays have shape $(T, 1)$.

- `interest_rate(...)` — small-open-economy real rate with a Schmitt-Grohé–Uribe debt-elastic premium; inputs `r_world` and lagged $NFA/Y \rightarrow r$.
- `production_firm(...)` — Cobb–Douglas final-goods block (legacy combined version with open- and closed-economy branches) $\rightarrow rK, w, K_tilde, Y$.
- `production_firm_prices(...)` — open-economy sub-block: factor prices from $r \rightarrow rK, w$.
- `production_firm_quantities(...)` — open-economy sub-block: K_tilde and Y

from the analytically closed L_{tilde} .

- `housing_investment(...)` — law of motion for the regional housing stock; $H_u, H_r \rightarrow IH_u, IH_r$.
- `construction_firm(...)` — construction sector zero-profit pricing and factor demands; $rK, w, IH \rightarrow q_u, q_r, K_u, L_u, K_r, L_r$ (optional convex adjustment cost).
- `mutual_fund(...)` — aggregates capital $K = K_{\text{tilde}} + K_u + K_r$ (and $r = rK - \delta$ in closed economy).
- `rental_sector(...)` — rental-investor FOC; $r, q_u, q_r \rightarrow$ regional rents f_u, f_r .
- `labor_market_close(...)` — analytically closes the labour market: $L_{\text{tilde}} = L_{\text{supply}} - L_u - L_r$.
- `market_clearing(...)` — housing-clearing residuals, investment, the resource constraint, and the external balance $NFA = A_{hh} - K$.

4 household_problem.py

The DC-EGM solver for the household problem (discrete housing choice + continuous savings), implementing Algorithms 1–2 of Iskhakov et al. (2017). The class `HousingModel` wraps a GE model.

- `__init__(model)` — initialises from a GE model (or standalone), exposing `par`, `ss`, `sol`.
- `solve_dcegm(prices_path)` — full backward DC-EGM solve over the life cycle at steady-state or time-varying prices; writes `sol.a`, `sol.c`, `sol.pr_choices`.
- `solve_dcegm_calendar_age_grid(prices_dict)` — single backward sweep over the whole (calendar-time $\times$ age) grid; returns policies for every (t, age) pair.
- `solve_dcegm_one_step(...)` — one backward DC-EGM step (used by the Jacobian machinery).
- `_solve_terminal_period(...)` — terminal-age policy: consume all cash-on-hand with a uniform choice prior.
- `_solve_non_terminal_period(...)` — interior DC-EGM step: EGM over savings

with the 4-way discrete housing choice and taste shocks.

- `_apply_upper_envelope(...)` — upper-envelope correction removing suboptimal points on non-monotone DC-EGM grids.
- `_utility` / `_marginal_utility` / `_inverse_marginal_utility` — CRRA utility over consumption and the housing service flow (with the kappa urban shifter) and its derivatives.
- `_choice_prob` / `_logsum` — softmax choice probabilities and the inclusive (log-sum) value.
- `_cash_on_hand_offsets` / `_cash_on_hand` — next-period cash-on-hand, including rent / mortgage costs and the one-time buy/sell and migration transaction costs.
- `solve_hh_backwards(model)` — module-level entry point called by `steady_state.py`; runs the solver and stores all policies in `model.sol`.

5 `steady_state.py`

Steady-state solution: household policies, the stationary distribution, aggregation, and price root-finding.

- `cash_on_hand_numba(...)` — Numba cash-on-hand for the 4 housing states (rent / mortgage mechanics).
- `build_transition_coo(...)` / `build_transition_coo_v2(...)` — build the COO sparse transition operator over (z , h , m); `v2` uses z' -conditional choice probabilities to capture the z' - h' correlation.
- `scatter_dcegm_forward(...)` — forward-propagate the distribution $D_{in} \rightarrow D_{out}$ without materialising a sparse matrix.
- `solve_hh_ss(model)` — iterate DC-EGM to convergence at steady-state prices.
- `simulate_hh_ss(model)` — compute the stationary distribution by the histogram method.
- `simulate_hh_ss_olg(model)` — OLG-consistent steady-state aggregator (cohort-by-cohort with age-specific policies).
- `household_ss(model)` — solve HH policies, simulate the distribution, and aggregate; returns `A_hh`, `C_hh`, `H_u_hh`, `H_r_hh`.

- `find_ss_indirect_housing(model)` — indirect steady-state step: calibrate technology so the given prices clear the regional housing markets.
- `find_ss_prices(model)` — root-find the factor prices (r , w) that clear all markets; housing prices are derived analytically from the construction zero-profit conditions.
- *Helpers*: `_pr_choices_from_pr_urban`, `_normalize_cohort_weights`, `_get_ss_cohort_weights`, `_get_newborn_distribution`, `_policy_arrays_from_endog` — cohort-weight handling, newborn distribution, and endogenous-to-regular grid interpolation.

6 `transition.py`

Transition-path computation and the sequence-space Jacobians.

- `evaluate_path_blocks(model)` — evaluates the full DAG along `model.path`; supports column-vectorised evaluation for fast finite differencing.
- `solve_hh_path(model)` / `simulate_hh_path(model)` — backward HH solve along the path / forward distribution simulation and aggregation.
- `_simulate_olg_path(model)` — OLG-correct non-linear forward pass (single calendar-age solve plus per-cohort simulation).
- `compute_age_specific_fake_news_matrices(...)` — age-specific fake-news matrices for one HH input-output pair (Bardóczy–Velásquez-Giraldo, 2025).
- `jacobian_from_fake_news_matrix(F)` — converts one fake-news matrix to one Jacobian via diagonal sums.
- `compute_age_specific_hh_jacobians(...)` — combines the two functions above into age-specific Jacobians.
- `compute_jac_hh_exact(model)` / `_forward_pass_outputs_exact(...)` — the exact sequence-space HH Jacobian by direct finite-difference perturbations.
- `compute_jacobians_complete(model)` — assembles `jac_hh`, the block Jacobians, and accumulates H_U along the DAG.
- `compute_path_jacobian(model)` / `compute_jacobians_around_path(model)` — recompute H_U (and `jac_hh`) linearised around the current path — the non-linear correction step.
- `_build_H_U_vectorized(...)` — builds H_U via one column-vectorised DAG sweep.

- `broyden_solver(...)` — quasi-Newton solver with rank-1 Broyden Jacobian updates.
- `find_transition_path(model, shocks, ...)` — top-level driver: solves the non-linear transition path via Broyden’s method.
- `compute_wealth_inequality(model)` — wealth-inequality time series (top shares, etc.) from the path distribution.
- `_set_unknowns` / `_get_errors` / `_evaluate_H` / `_evaluate_H_nonlinear` — map the solver vector to/from path arrays, with hard terminal anchoring of the unknowns.
- *Helpers:* policy-array I/O (`_write_policy_*`, `_policy_arrays_from_endog`), cohort-weight and newborn-distribution handling, shifted-price construction, and the Numba kernel `_simulate_cohort_njit`.

7 calibration.py

Minimum-distance calibration of structural parameters to data targets.

- `compute_ss_moments(model)` — computes the calibration moments from a solved steady state; returns a dict keyed to the target specification.
- `load_targets(data_path, target_spec, base_year)` — reads the empirical target values for each moment from an `xlsx` file.
- `calibrate(model, param_spec, target_spec, ...)` — calibrates the model to base-year data via Nelder–Mead minimum-distance estimation.
- `report(...)` — prints a calibration summary or a model moments-vs-data / steady-state-levels table.
- *Helpers:* `_ss_validity_penalty` (quadratic penalty for infeasible steady states), `_compute_wealth` (top-10% and bottom-50% net-wealth shares).

8 simulation.py

Partial-equilibrium OLG wealth-distribution analysis: prices are treated as exogenous (no market clearing) and the household block is simulated forward.